

JOINS e Relacionamentos

Modelando dados conectados em PostgreSQL + sqlc

DIM0547 — Sprint 2 · Material de apoio

Prof. Fernando · UFRN · 2026.1

Hoje

O que vamos ver

- Por que separar em duas tabelas
- Foreign keys e `ON DELETE CASCADE`
- 1:N na prática — contatos com telefones
- LEFT JOIN: o que vem como NULL
- `pgtype.*`: o "Maybe" do Go
- Agregação no Go vs subquery no SQL
- N+1: o anti-padrão clássico

Pré-requisitos

- Aula anterior (PostgreSQL + sqlc)
- `:many`, `:one`, `:exec`
- Como `sqlc generate` mapeia tipos

Por quê este material

O Ex04 da Lista 4 envolve relacionamento 1:N. Antes de codar, vamos entender o modelo.

0 problema: contato com vários telefones

Ingênuo: campo único

```
CREATE TABLE contacts (  
  id SERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL,  
  phone TEXT -- ?  
);
```

Como representar 3 telefones? "+55 84 1, +55 84 2, +55 84 3" ?

Problemas

- **Busca quebra:** WHERE phone = '+55 84 1'
não casa com "+55 84 1, +55 84 2"
- **Tamanho indefinido:** alguns têm 1, outros 5
- **Sem metadados:** e se eu quiser saber qual é "casa", "celular", "trabalho"?
- Está **violando a 1ª forma normal** — célula com lista

Quando a resposta é "vários", a resposta é **outra tabela**.

Modelagem 1:N

```
CREATE TABLE contacts (  
  id      SERIAL PRIMARY KEY,  
  name    TEXT NOT NULL,  
  email   TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE phones (  
  id          SERIAL PRIMARY KEY,  
  contact_id  INTEGER NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,  
  label       TEXT NOT NULL,          -- 'casa', 'trabalho', 'celular'  
  number      TEXT NOT NULL  
);
```

Lê-se: cada telefone pertence a **um** contato (`REFERENCES contacts(id)`). Um contato pode ter **muitos** telefones — não há limite aqui.

A direção da seta vai do **lado N** para o **lado 1**: a foreign key fica em `phones`, apontando para `contacts`.

Foreign key: garantia no banco

```
contact_id INTEGER NOT NULL REFERENCES contacts(id)
```

Sem FK	Com FK
Posso inserir <code>phone(contact_id=999)</code> mesmo se <code>contacts(id=999)</code> não existe	INSERT falha — banco rejeita
Posso deletar <code>contacts(id=1)</code> deixando <code>phones</code> órfãos	DELETE falha — banco rejeita
Aplicação tem que validar tudo no código	Garantido pelo banco mesmo se a app errar

A integridade referencial é uma **invariante do banco**. Mesmo com 3 microsserviços batendo no mesmo banco, ninguém consegue criar um telefone órfão.

ON DELETE: o que fazer com os filhos

```
contact_id INTEGER NOT NULL  
REFERENCES contacts(id)  
ON DELETE CASCADE
```

Política	O que acontece ao deletar o pai
CASCADE	Filhos somem juntos (escolhemos isso)
SET NULL	contact_id vira NULL nos filhos (precisa permitir NULL)
RESTRICT (padrão)	DELETE do pai falha se houver filhos
NO ACTION	Igual ao RESTRICT, mas verificado ao final da transação

CASCADE faz sentido aqui: telefone sem contato não tem significado. Em outros domínios (**orders** → **customers**), você prefere **RESTRICT** — não pode apagar um cliente que tem histórico.

JOIN: combinando linhas de duas tabelas

A pergunta: "Quero todos os contatos com seus telefones."

```
SELECT
    c.id, c.name,
    p.id, p.label, p.number
FROM contacts c
JOIN phones p ON p.contact_id = c.id;
```

O que o banco faz: para cada linha de `contacts`, procura linhas em `phones` onde `phone.contact_id = contact.id`. Quando encontra, "cola" as colunas das duas tabelas numa linha só.

contacts		phones			resultado do JOIN					
id	name	id	contact_id	label	number	c.id	c.name	p.id	p.label	p.number
1	Maria	10	1	casa	1111	→ 1	Maria	10	casa	1111
2	João	11	1	cel	2222	1	Maria	11	cel	2222
		12	2	casa	3333	2	João	12	casa	3333

INNER JOIN vs LEFT JOIN

-- INNER JOIN (ou só JOIN): só linhas que casam dos DOIS lados

```
SELECT * FROM contacts c JOIN phones p ON p.contact_id = c.id;
```

-- → contato sem telefone NÃO aparece

-- LEFT JOIN: TODAS as linhas da esquerda, com NULL onde não casa

```
SELECT * FROM contacts c LEFT JOIN phones p ON p.contact_id = c.id;
```

-- → contato sem telefone aparece, mas com p.* todos NULL

INNER JOIN

c.id	name	p.id	label	number
1	Maria	10	casa	1111
1	Maria	11	cel	2222

LEFT JOIN

c.id	name	p.id	label	number
1	Maria	10	casa	1111
1	Maria	11	cel	2222
2	João	NULL	NULL	NULL ← !

Se o objetivo é listar **todos** os contatos (mesmo sem telefones), **LEFT JOIN é obrigatório**. INNER JOIN esconde quem não tem telefone.

0 resultado é "achatado"

SQL retorna **uma linha por combinação**. Maria com 3 telefones vira **3 linhas**, repetindo `c.id` e `c.name`.

c.id	c.name	p.id	p.label	p.number
1	Maria	10	casa	1111
1	Maria	11	celular	2222
1	Maria	12	trabalho	3333
2	João	NULL	NULL	NULL

O cliente da API quer ver:

```
[
  {"id": 1, "name": "Maria",
   "phones": [{"label": "casa", ...}, {"label": "cel", ...}, {"label": "trab", ...}]},
  {"id": 2, "name": "João", "phones": []}
]
```

Existe um descasamento de formas: relacional é **plano**, JSON é **aninhado**. Alguém tem que aninhar — geralmente o Go.

A query do Ex04

```
-- name: ListContactsWithPhones :many
SELECT
  c.id, c.name, c.email, c.created_at,
  p.id    AS phone_id,
  p.label,
  p.number
FROM contacts c
LEFT JOIN phones p ON p.contact_id = c.id
ORDER BY c.id ASC, p.id ASC;
```

Dois detalhes importantes:

- **AS phone_id** — sem o alias, **p.id** colidiria com **c.id** no struct gerado pelo sqlc. O sqlc avisa, mas é boa prática nomear explicitamente.
- **ORDER BY c.id, p.id** — agrupar por contato exige que linhas do **mesmo** contato venham **adjacentes**. Sem ORDER BY o Postgres pode entregar fora de ordem.

0 que o sqlc gera para essa query

```
type ListContactsWithPhonesRow struct {
    ID          int32      // c.id          – NOT NULL no schema
    Name        string     // c.name        – NOT NULL
    Email       string     // c.email       – NOT NULL
    CreatedAt   time.Time  // c.created_at  – NOT NULL
    PhoneID     pgtype.Int4 // p.id          – pode vir NULL pelo LEFT JOIN
    Label       pgtype.Text // p.label       – pode vir NULL
    Number      pgtype.Text // p.number      – pode vir NULL
}
```

Observação crucial: na tabela `phones`, todas as colunas são `NOT NULL`. Mas pelo **LEFT JOIN**, quando um contato não tem telefones, essas colunas **vêm NULL** — porque não existe linha em phones para juntar.

sqlc analisa o JOIN e detecta isso. É por isso que `pgtype.Int4` aparece em vez de `int32`. **A nullabilidade não vem só do schema — vem da query também.**

0 Maybe do Go: pgtype.*

Go não tem tipo `Maybe` / `Optional`. Para representar "pode ser NULL", o pgx usa um struct com dois campos:

```
type pgtype.Int4 struct {
    Int32 int32 // o valor (se Valid)
    Valid bool   // true = tem valor; false = era NULL
}

type pgtype.Text struct {
    String string
    Valid  bool
}

type pgtype.Timestamptz struct {
    Time  time.Time
    Valid bool
}
```

```
if row.PhoneID.Valid {
    fmt.Println(row.PhoneID.Int32) // tem valor
} else {
    fmt.Println("era NULL")
}
```

`Valid` é o "tag" do Maybe. Não dá para esquecer de checar — esse é o ponto. Acessar `.Int32` quando `Valid == false` te dá o **zero value** (0) em vez de pânico, mas semanticamente é errado: 0 é uma resposta legítima.

Agregação no Go: o algoritmo

A entrada é uma lista de linhas achatadas; a saída é uma lista de contatos com array de telefones.

ENTRADA (rows)					SAÍDA (result)
c.id	name	p.id	label		[{id:1, name:"Maria", phones:[
1	Maria	10	casa	→	{id:10, label:"casa"},
1	Maria	11	cel		{id:11, label:"cel"}]
2	João	NULL	NULL		{id:2, name:"João", phones:[]}]

Algoritmo (uma passada):

1. Mantenha um **mapa** `contactID → índice no resultado`.
2. Para cada linha:
 - Se nunca vi esse `contactID`, crio um `ContactWithPhones` com `phones: []` e adiciono ao resultado.
 - Se a linha tem `PhoneID.Valid`, anexo o telefone ao array do contato correspondente.
3. Pronto — uma passada, $O(N)$.

Agregação no Go: o código

```
result := []ContactWithPhones{}
index := map[int32]int{} // contactID → posição em `result`
for _, row := range rows {
    i, exists := index[row.ID]
    if !exists {
        i = len(result)
        result = append(result, ContactWithPhones{
            ID:      row.ID,
            Name:    row.Name,
            Email:   row.Email,
            Phones: []PhoneOut{}, // nunca nil – para serializar [] em vez de null
        })
        index[row.ID] = i
    }
    if row.PhoneID.Valid { // !! checagem do Maybe
        result[i].Phones = append(result[i].Phones, PhoneOut{
            ID:      row.PhoneID.Int32,
            Label:   row.Label.String,
            Number:  row.Number.String,
        })
    }
}
```

Por que slice + mapa em vez de só mapa? Para preservar a **ordem** definida pelo **ORDER BY** da query. Map iteration em Go é não-determinística por projeto.

Detalhes operacionais do Ex04

O stub que entra em pânico

`ex04/internal/db/contacts.sql.go` já vem com **stubs** para `CreatePhone`, `ListPhonesByContact` e `ListContactsWithPhones`. São funções que entram em **panic** para o template **compilar** sem você ter feito o trabalho.

Se rodar o teste antes de:

1. preencher `db/queries/contacts.sql`
2. rodar `sqlc generate`
3. **comitar** `internal/db/contacts.sql.go`

você verá:

```
panic: ListContactsWithPhones não foi
gerada – preencha db/queries/contacts.sql
e rode `sqlc generate`
```

O CI **não** roda `sqlc generate`. O autograding lê o que estiver no repo.

`phones: []` vs `phones: null`

JSON em Go: `nil` slice serializa como `null`. Os testes do Ex04 verificam que contatos sem telefones têm `"phones": []`, não `null`.

```
// ✗ Phones inicializado como nil
result = append(result, ContactWithPhones{
    ID:    row.ID,
    Name:  row.Name,
})
// → "phones": null no JSON

// ✓ Inicializa como slice vazia
result = append(result, ContactWithPhones{
    ID:    row.ID,
    Name:  row.Name,
    Phones: []PhoneOut{}, // <-
})
// → "phones": [] no JSON
```

Mesma armadilha do `listContacts` no Ex01.

E o problema N+1?

Alternativa **ingênua**: 1 query para listar contatos + 1 query por contato para buscar telefones.

```
contacts, _ := q.ListContacts(ctx)
for _, c := range contacts {
    phones, _ := q.ListPhonesByContact(ctx, c.ID) // ← N queries!
    ...
}
```

```
100 contatos = 1 + 100 = 101 queries
1000 contatos = 1001 queries
```

Cada query é um round-trip à rede do banco (~1-5ms cada). 1000 contatos = potencialmente 1-5 segundos só esperando.

Esse é o **problema N+1** — clássico, conhecido por nome. A solução padrão é uma única query com JOIN (que é o que ListContactsWithPhones faz: **1 query**, sem importar quantos contatos).

Quando JOIN, quando N+1?

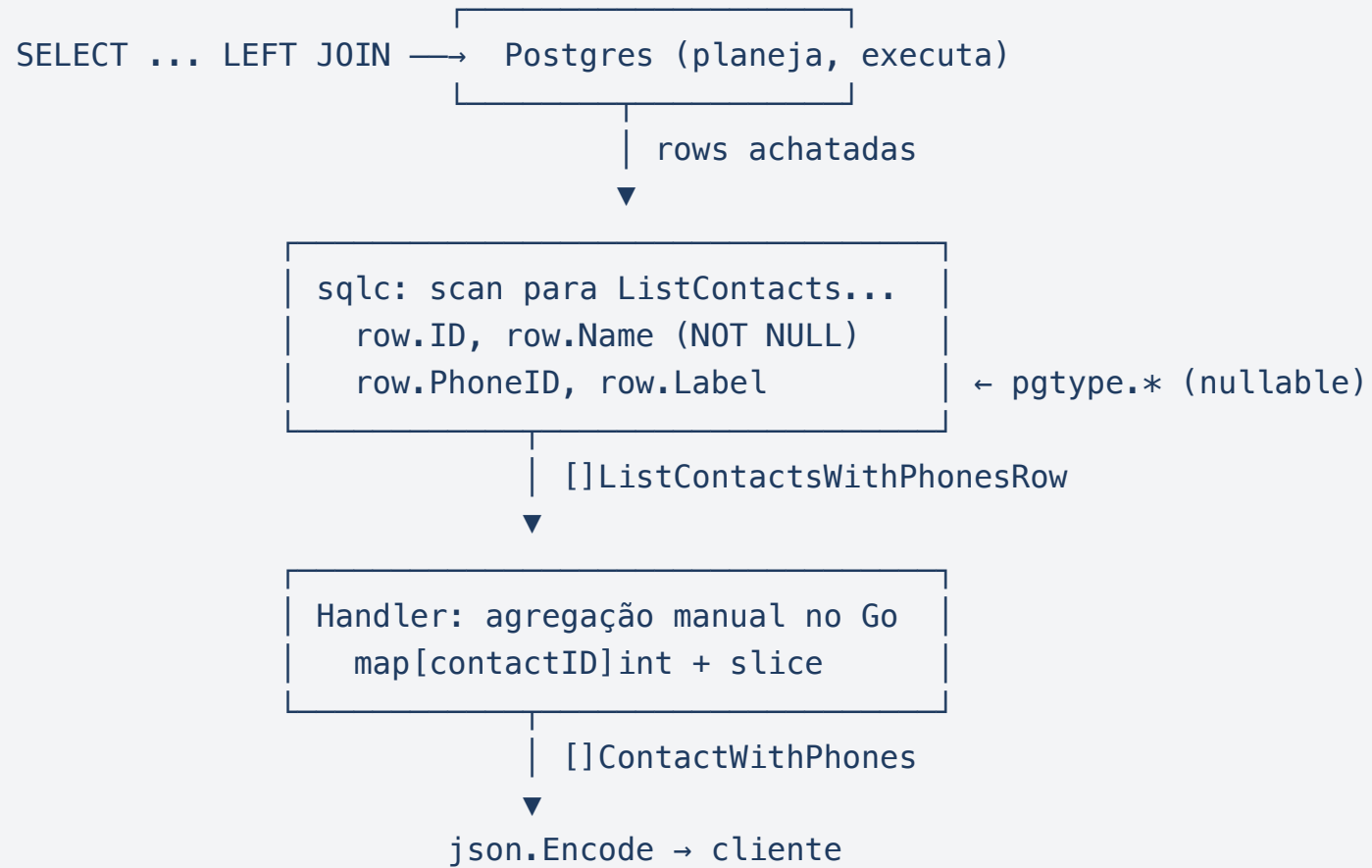
Situação	Estratégia	Por quê
Listar todos com filhos	JOIN	Evitar N+1
Detalhar um item específico	N+1 (1+1)	É só uma query extra — simples e legível
Listar com filhos OPCIONAIS (carregamento lazy no front)	Endpoints separados	Front pede só o que precisa
Filhos com paginação independente	Endpoints separados	LIMIT no JOIN é traiçoeiro

No Ex04 você tem os dois:

- `GET /contacts/{id}/phones` — caso 1+1, simples
- `GET /contacts-with-phones` — caso JOIN, listagem completa

Ambos existem por motivo pedagógico: ver os dois trade-offs lado a lado.

Recapitulando: o caminho do dado



Referências

PostgreSQL

- `CREATE TABLE` ([Foreign Keys](#)) — sintaxe completa de FK e `ON DELETE`
- [Tutorial: Foreign Keys](#)
- `SELECT` com [JOINS](#)

sqlc

- [Type-mapping](#) — PostgreSQL → Go
- [Configuration](#) — `emit_pointers_for_null_types`

Como entregar – Lista 4

Entrega **individual** via GitHub Classroom. Prazo **19/05 (ter) às 23:59**.

Fluxo de entrega

```
# após preencher SQL e rodar sqlc generate
git add .
git commit -m "ex04: JOIN com phones"
git push
```

Cada push dispara o autograding. Acompanhe na aba **Actions** do seu repositório do Classroom.

No Ex04, lembre de `git add internal/db/contacts.sql.go` — o autograding **não roda** `sqlc generate`.

Pontuação

Exercício	Pontos
Ex01 — sqlc básico	25
Ex02 — Repository	25
Ex03 — Filtros + paginação	25
Ex04 — JOIN 1:N	25
Total	100

Política de atraso

- Até 1 dia: **-20%**
- Até 3 dias: **-50%**
- Após 3 dias: **não aceito**

A Lista 4 é avaliação **individual** — não compartilhe código com colegas. A Sprint 2 do projeto, em grupo, é entrega separada (mesmo prazo, mas via SIGAA).

Próximos passos

- **Lista 4 / Ex04** — implementa exatamente esses conceitos. O `ex04/README.md` tem 4 seções "Conceito" expandindo os pontos sutis (LEFT JOIN, `pgtype.*`, agregação no Go, N+1). Volte a esses slides e ao README quando travar.
- **Tabela "Erros comuns"** no `ex04/README.md` — antes de pedir ajuda, confira se seu sintoma está lá.
- **Pergunta a se fazer durante o exercício**: "Se eu remover o `LEFT` em `LEFT JOIN`, o que muda no resultado e nos tipos gerados?"
- **Bônus opcional**: reescrever `ListContactsWithPhones` usando `json_agg` — agregação no banco em vez de no Go.

Acompanhamento online: **14/05 (qui) 13:00–14:40** — venham com suas dúvidas.